

Redesigned Competitive Mode Layout (Android 15)

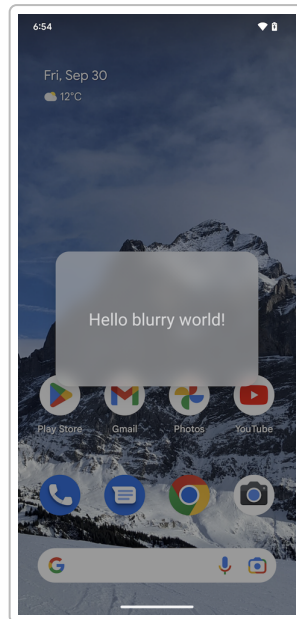
Overview: The Competitive Mode chess screen will be modernized with Android 15 (API 35) features, while preserving the XML-based View architecture. We will implement a **glassmorphic** UI motif – translucent frosted-glass panels and floating controls – atop a vibrant backdrop. This involves using GPU blur effects, Material 3 theming, and enhanced navigation/transitions. The result will be an elegant, immersive interface that takes full advantage of a modern Android 15 flagship's capabilities.

Glassmorphic Design with RenderEffect Blur

Frosted Glass Panels: Key UI containers (e.g. the top player info bar and bottom controls panel) will have a **frosted glass** appearance – translucent with blurred background content. We achieve this by layering a semi-transparent shape and applying a **GPU blur** via `RenderEffect`. Android 12+ introduced `RenderEffect.createBlurEffect()` for real-time blurring of Views ¹. In code, after inflating the layout, we can do:

```
if (Build.VERSION.SDK_INT >= 31) {  
    val blurRadius = 20f  
    val blurEffect = RenderEffect.createBlurEffect(blurRadius, blurRadius,  
        Shader.TileMode.CLAMP)  
    frostedPanelView.setRenderEffect(blurEffect) // Apply blur to panel  
    background 2  
}
```

This will blur the content **behind** the target view (the panel) when drawing it. Each frosted panel uses a custom drawable background – for example, a rounded rectangle shape with a translucent white fill (e.g. `#80FFFFFF` for ~50% opacity white) and maybe a subtle white outline or inner glow to enhance the “glass” illusion. The translucent layer combined with blur produces the acrylic frosted effect ³ ⁴.



Example of a translucent blurred card (“Hello blurry world!”) creating a frosted glass effect over a colorful background ⁵ ⁶.

Gradient Background: To make the glass effect pop, the app’s root background will be a high-contrast **gradient** or imagery instead of a flat color. A rich, colorful backdrop (for example, a subtle blue-purple gradient or a blurred chess-themed image) will show through the translucent panels. According to Material and glassmorphism guidelines, a **vibrant background** accentuates the blur effect ⁷. We can supply a drawable (e.g. `bg_gradient.xml`) as the `android:background` for the root container. This gradient might gently vignette toward edges, ensuring foreground text remains legible. Furthermore, the blur radius and panel opacity should be tuned together for readability – e.g. using ~20px blur radius for depth-of-field and pairing it with a translucent overlay color ⁸ ⁴. Avoid excessive blur (Google recommends staying ≤ 150 px radius to prevent performance hits ⁸). Given our target is a Galaxy S23-class device, moderate blur (around 20–50px) is feasible without stutter.

Subtle Shadows/Glows: The frosted panels will also have subtle **elevation shadows** or glows to stand out. We can leverage Material 3 elevation for dynamic shadows – for example, giving the top info bar and bottom control panel a small elevation (4dp–8dp) to cast a soft shadow on the background. This not only adds depth but also a “halo” glow around translucent edges. Additionally, a faint white **stroke** on panel borders (e.g. 1px with 15% opacity white) can simulate light catching the glass edges. All these details contribute to a cohesive glassmorphic aesthetic.

Material 3 Theme Integration

We will update the app theme to **Material You (Material 3)** for modern widgets and styling. Using the Material Components library (MDC 1.10+), we create a theme in `styles.xml` inheriting from a Material3 theme (e.g. `Theme.Material3.Light.NoActionBar`) as a base). Material 3 brings updated default colors, typography, and components that will make our UI feel up-to-date. Some integration steps:

- **Color Scheme:** Define a Material 3 color scheme (or enable dynamic color). Since our design uses a custom palette (wood and bronze tones in the legacy design), we can map those to the Material3 roles (primary, secondary, surface, etc.), or directly use the Material 3 **neutral** tones for

the glass panels. For example, the frosted panels might use `surfaceVariant` with 30% opacity as background, and text icons use `onSurface` for contrast. Material 3 also supports **light vs dark** themes – we can ensure the design works in both (perhaps preferring a dark translucent background for system bars).

- **Typography & Icons:** We continue using the distinctive serif font for player names as in the original, but Material 3's Typography styles can be applied for consistency in other text (e.g. body, labels). The VS emoji and other icons can remain, or we could use Material Icons for clarity (e.g. a standard VS icon if available). Floating action buttons (like the "Game Analysis" button) will use Material3 styling by default (with the `FloatingActionButton` from MDC).
- **Material Widgets:** We replace any basic widgets with Material components where appropriate. For instance, the bottom control buttons can be `MaterialButton` with the text+icon (if needed) to get updated ripples and styling. We can use `ShapeableImageView` or `MaterialCardView` if we want to add rounded corners or outlines easily. The original layout's use of `CardView` for coach selection, etc., can migrate to `MaterialCardView` with the new theme, gaining the Material3 default corner radius (which we can override). Overall, Material3 ensures **consistency** and compatibility with Android 15 features (it even ties into predictive back gestures in MDC 1.10 ⁹).
- **Styles and Drawables:** We will prepare new style resources for the glass design. For example, a style `GlassPanel1` could set `android:background="@drawable/glass_panel_bg"` (the frosted shape drawable) and apply it to our header and bottom panels. A `GlassButton` style might define a transparent or mildly tinted background and `textColor` per Material3 guidelines (perhaps using `MaterialButton` with `android:backgroundTint` set to a semi-transparent accent). We'll also define the gradient drawable (`gradient_background.xml`) used in the theme as the Activity background. These resources ensure the look is easily maintainable and consistent across the app.

Edge-to-Edge UI & System Bar Adaptation

To enhance immersion, the layout will go **edge-to-edge**, drawing behind the status and navigation bars. Android 15 enforces edge-to-edge content for apps targeting SDK 35 ¹⁰, so we'll embrace this design fully. Key steps:

- **Transparent System Bars:** In the app theme, set the status bar and navigation bar to transparent:

```
<item name="android:statusBarColor">@android:color/transparent</item>
<item name="android:navigationBarColor">@android:color/transparent</item>
<item name="android:windowDrawsSystemBarBackgrounds">true</item>
```

This allows our content (gradient background and frosted panels) to extend behind the system bars. On Android 15+, the system **gesture bar** is always translucent, so content will show underneath by default ¹¹. We must therefore handle UI inset padding manually.

- **Handling Insets:** We remove legacy `android:fitsSystemWindows="true"` from layouts (the current layout uses it on the root `LinearLayout`, which would prevent drawing behind bars). Instead, we use the `WindowInsets` API to adjust padding only where needed. For example, we

can add top padding equal to the status bar height to the header bar, and bottom padding to the bottom panel, so that content isn't obscured by the bars. Material Components can automatically handle some insets (e.g. navigation rail, FAB alignment), but in our custom layout we'll do:

```
WindowCompat.setDecorFitsSystemWindows(window, false) // let content extend
ViewCompat.setOnApplyWindowInsetsListener(rootView) { view, insets ->
    val sysBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
    headerContainer.setPadding(headerContainer.paddingLeft, sysBars.top,
                              headerContainer.paddingRight, 0)
    bottomPanel.setPadding(bottomPanel.paddingLeft, 0,
                           bottomPanel.paddingRight, sysBars.bottom)
    WindowInsetsCompat.CONSUMED
}
```

This ensures the top panel's content is pushed below the status bar area, and the bottom controls stay above the navigation gesture area. On a Galaxy S23 Ultra with gesture nav, for example, the bottom `sysBars.bottom` inset might be ~80px, providing enough gap. (If the device uses 3-button nav, the bar is translucent by default on SDK 35 ¹², so similar handling applies.)

- **Adaptive System Bar Icons:** With dynamic backgrounds, we must keep status bar icons legible. Android provides the `WindowInsetsController` to control icon color. For instance, if our top area is a light translucent panel, we'll use dark icons (`isAppearanceLightStatusBars = true`). Conversely, if showing a dark background, use light icons. We can toggle this at runtime if the UI's backdrop changes, but since our design's background is mostly static (say a dark bluish gradient), we'll configure it once (e.g. in theme set `<item name="android:windowLightStatusBar">false</item>` for light text, assuming a dark background). The same goes for navigation bar icon contrast if needed. This **adaptive coloring** fulfills the "chameleon" effect where system bars seamlessly blend with the app ¹³.
- **Immersive Touches:** The chessboard view itself can now extend fully, possibly even reaching under the status bar for a dramatic look (imagine the board graphics going to the very top, with the frosted header overlapping it slightly in a translucent manner). We will be careful that interactive elements (e.g. a pause button at top or the edge of the board) are not too close to gesture edges (we use `systemGestureInsets` if necessary ¹⁴ to offset touch targets from the very edge). Overall, the edge-to-edge design makes the game feel full-screen and modern.

Predictive Back Navigation

Android 15 introduces **predictive back gesture** animations system-wide, providing a visual preview of the destination when users swipe back ¹⁵. We will ensure Competitive Mode supports this smoothly:

- **Enable Predictive Back:** As of Android 14+, apps must opt-in to predictive back if they intercept back events. In our case, we likely use the default back (to leave the game screen) so minimal changes are needed. We will confirm that the manifest (or Activity) hasn't disabled `OnBackPressedCallback`. By default, targeting SDK 35 means predictive back is **enabled by default** for the app ¹⁶. We just need to avoid legacy back overrides. In the manifest, we can explicitly set `android:enableOnBackPressedCallback="true"` on the application or `CompetitiveMode` activity (though not strictly required if using AndroidX back dispatcher). This ensures the system knows our activity is ready for the new back handling.

- **Using OnBackPressedDispatcher:** If the app currently uses `onBackPressed()` in the Activity or a custom KeyEvent handler, we will migrate that to either the **AndroidX OnBackPressedDispatcher** or the platform **OnBackPressedCallback** API ¹⁷. This is important because intercepting the legacy back event could block the predictive animation. By using the modern callbacks, the system can coordinate the gesture with our logic. For example, in `onCreate()` of `CompetitiveModeActivity`:

```
onBackPressedDispatcher.addCallback(  
    this,  
    object : OnBackPressedCallback(true) {  
        override fun handleOnBackPressed() {  
            // Custom behavior (if any), then:  
            remove() // ensure system can proceed with normal back  
            onBackPressedDispatcher.onBackPressed() // call default  
        }  
    }  
)
```

If we have no special back behavior (just go to previous screen), we might not need any code at all – the default will work with predictive animations out-of-the-box. The main point is not to disable the predictive system. (For activities that perform a **confirm exit** dialog, etc., we'd incorporate that with the new APIs so the swipe can be completed if the user confirms.)

- **Back Animation Preview:** With everything enabled, when the user swipes back from the Competitive Mode screen, they will see the underlying activity (perhaps a Main Menu or Launcher home screen) start to peek in ¹⁸. For cross-task back (back to home launcher), Android will show the home screen behind our app during the gesture. For in-app back (e.g. from game to a menu activity), the previous Activity's UI can be previewed. We should ensure that previous screen is ready to be drawn. If using multiple activities, consider enabling **Cross-activity transitions** (described next) so the return transition is smooth in concert with the back gesture.

Overall, with predictive back implemented, our app will feel **responsive and intuitive** on Android 15, matching user expectations of the platform navigation.

Shared Element Transitions

We will enhance transitions between screens by using **shared element animations** – for example, smoothly transitioning the chessboard view or a piece image between the Competitive Mode and other activities (like an analysis screen or a game list). Shared element transitions make the UI feel connected and polished. Implementation steps:

- **Define Shared Elements:** Identify which views will be common between activities. A prime candidate is the chessboard itself. Suppose when the user taps “Game Analysis” (the FAB), we launch `GameAnalysisActivity` which also displays the final board position. We can make the `ChessBoardView` in Competitive Mode and the one in Analysis share the same transition name:

```
<com.example.ChessBoardView
    android:id="@+id/competitiveChessBoardView"
    android:transitionName="chessboard" ... />
```

and in analysis layout:

```
<com.example.ChessBoardView
    android:id="@+id/analysisBoardView"
    android:transitionName="chessboard" ... />
```

This tags them for sharing.

- **Enable Content Transitions:** In the styles or manifest, enable window content transitions for these activities. For instance, set the activity theme to include:

```
<item name="android:windowContentTransitions">true</item>
```

(Material3 themes often have this on by default.) Also ensure

`android:windowAllowSharedElementsTransition` is true (usually covered by the above).

We typically also disable the shared elements overlay for a continuous effect:

```
getWindow().setSharedElementsUseOverlay(false);
```

(This prevents the shared view from duplicating during transition.)

- **Launch with Transition:** When starting the new Activity, use the AndroidX Activity options to specify the shared element. For example:

```
val options = ActivityOptionsCompat.makeSceneTransitionAnimation(
    this@CompetitiveModeActivity,
    competitiveChessBoardView,
    "chessboard" // transitionName
)
startActivity(intent, options.toBundle())
```

This hands off the shared element to the system for animation. We can also include multiple elements (e.g. a player avatar or a trophy icon) by pairing them in the options.

- **Use Material Container Transform (optional):** Given we are using Material Components, we can leverage **MaterialContainerTransform** for a more elaborate shared transition. This is a Material motion pattern that morphs a container between activities. For example, if we had a card list of games that expands into the game screen, a container transform would animate size and shape changes smoothly. To use it, we would set the shared element callback in each activity:

```
// In CompetitiveModeActivity (exit)
window.sharedElementExitTransition = MaterialContainerTransform().apply
{ ... }
setEnterSharedElementCallback(MaterialContainerTransformSharedElementCallback())
// In GameAnalysisActivity (enter)
window.sharedElementEnterTransition =
MaterialContainerTransform().apply { ... }
setEnterSharedElementCallback(MaterialContainerTransformSharedElementCallback())
```

Material's callbacks handle shape rounding and elevation matching during the transition ¹⁹ ²⁰ . This would give a smooth morphing effect of the board “growing” into the analysis screen, for instance. We can tune the transform (duration, interpolator) to match the aesthetic (fast and ease-out for a snappy feel).

By implementing shared element transitions, we add a layer of **polish**: navigating between the Competitive Mode and other parts of the app will feel fluid, as elements don't abruptly disappear but gracefully transform. This is especially effective on a high-performance device where complex transitions run at 60+ FPS.

Implementation Deliverables

1. XML Layout: The new `activity_competitive_mode.xml` will incorporate the changes. We maintain the overall structure (header, board area, controls) but update attributes:

- The root uses a `CoordinatorLayout` or `ConstraintLayout` for flexibility (we can stick with `LinearLayout` as in the original for simplicity, but `ConstraintLayout` could help position floating buttons easily). It will have `android:background="@drawable/bg_gradient"` for the gradient, and no `fitsSystemWindows`.
- The header container (`competitiveHeaderContainer`) becomes a frosted glass panel. For example:

```
<LinearLayout
    android:id="@+id/competitiveHeaderContainer"
    style="@style/GlassPanel" <!-- new style for frosted panels -->
    android:orientation="horizontal"
    android:padding="8dp">
    ...
</LinearLayout>
```

The `GlassPanel` style would set the background drawable (`glass_panel_bg`) and perhaps default elevation. Inside this, we might remove the solid sub-panel backgrounds (formerly `@color/chess_light_square` and `@color/bronze`). Instead, text sits on the unified translucent backdrop. If needed, we can still subtly differentiate sides by applying a slight tint. For example, the player name side could have a foreground drawable that adds a very light white tint, and the master side a very light golden tint – both extremely transparent so the overall look remains glassy. The “VS” `TextView` stays in the middle with perhaps bold text and no background. All text colors should be checked against the new background for contrast – likely

we'll use `@color/text_primary` (white or near-white) since the panel is semi-transparent and the background may be dark behind.

- The chessboard area (inside the `FrameLayout`) remains largely the same. The board view (`ChessBoardView`) is already centered and sized to full available space. We will give it a `android:transitionName="chessboard"` to facilitate transitions. The evaluation bar on the left can also be styled minimally (perhaps make it semi-transparent as well). The board itself might not need blur (the board is the main content to remain sharp). However, any overlay (like highlighting last move, or a glowing effect when AI is "thinking") can be accented – e.g., the `masterThinkingProgressBar` could be a **Material 3 circular indicator** styled with our accent color.
- The bottom control panel (`LinearLayout` with 7 buttons) is given the same `@style/GlassPanel` background for consistency. We'll also set `android:orientation="vertical"` and perhaps a corner radius on the drawable so its top edges are curved (since it spans the bottom width). Inside, each `Button` can be switched to `MaterialButton` with styles like `Widget.Material3.Button` for updated appearance. We might use the icon+text styling for the "🔊 TTS" and "🧠 AI" buttons so the emoji is more of an icon. These buttons' background tints (amber, bronze, etc.) in the old design could be retained, but to align with the new scheme, we might standardize on one accent color or use tonal variants. For example, use Material3's primary or secondary color for all control buttons to reinforce a cohesive palette, and rely on their icons/labels to indicate function (color-coding each button is less necessary in a modern design). The buttons can still have slight transparency if desired (we can set a color state list with some alpha).

We will produce the complete XML with these updates, ensuring all IDs and references match the existing logic (so the code-behind can remain largely unchanged apart from new UI calls for blur, etc.).

2. Drawables & Styles: Supporting resources will include:

- `glass_panel_bg.xml`: a shape drawable with `<corners android:radius="12dp"/>` (for rounded corners), `<solid android:color="#80FFFFFF"/>` (white at 50% opacity as base). Optionally a `<stroke android:width="1dp" android:color="#4DFFFFFF"/>` (30% opacity white) for a faint border. We might also add an inner shadow/glow by layering multiple shapes (or let the View's elevation handle shadow). This drawable will be used as the background for all frosted panels.
- `bg_gradient.xml`: a layer-list or shape that defines the background gradient. For example, a vertical gradient from a deep blue (`#1A237E`) to a rich purple (`#4A148C`) or any theme-appropriate colors (taking inspiration from the "wood" theme, perhaps a warm gradient from a dark brown to a desaturated gold). We ensure it's not too bright at the top, so status bar icons (white) have contrast.
- Styles in `themes.xml` or separate `styles.xml`: We add `GlassPanel` style for containers as mentioned. Also `GlassButton` if needed for translucent button backgrounds (though using `MaterialButton` with default style might suffice). We also adjust the app theme: set `EdgeToEdge` preferences (transparent bars, etc.), use `Theme.Material3` parent, and include our color palette customization (via `materialColorPrimary`, etc., or a M3 color XML if using dynamic scheme).

3. Code for Blur & Edge-to-Edge: In the `CompetitiveModeActivity` (Java/Kotlin), we will initialize the blur effects and edge-to-edge. After `setContentView`, do:

- `WindowCompat.setDecorFitsSystemWindows(window, false)` to enable drawing behind bars.

- Apply `RenderEffect` to panels: e.g.

`findViewById<View>(R.id.competitiveHeaderContainer).setRenderEffect(blurEffect)` and similarly for the bottom panel. (We wrap this in an SDK check for API 31+ as shown earlier.) If we want different blur radii for different elements, we can adjust (perhaps a slightly higher blur on the background of a large panel to really diffuse what's behind).

- Set up **OnBackPressedCallback** (or ensure the dispatcher is used). For example, in an Activity written in Kotlin targeting SDK 35, we could simply rely on the default back handling. But if needed:

```
onBackPressedDispatcher.registerOnBackPressedCallback(
    OnBackPressedDispatcher.PRIORITY_DEFAULT
) {
    finish() // or custom action on back gesture release
}
```

This uses the new API directly. Alternatively, if using AndroidX, the `onBackPressedDispatcher` as discussed earlier is fine (especially if other parts of app are already using it). The main deliverable is that back is handled in compliance with predictive nav. We'll document this in code comments.

- Initiate shared element transition if coming from another activity: If `CompetitiveModeActivity` is launched with a shared element (say from a game list), we call `supportPostponeEnterTransition()` in `onCreate` until the layout and chessboard are fully drawn, then `supportStartPostponedEnterTransition()`. This ensures the shared element animation (if any) starts only when ready. Similarly, when launching out, use `ActivityOptions` as described. We'll also update the manifest to declare `android:sharedElementReentryTransition` if we want the reverse transition when coming back.

4. Manifest Config: We will update `AndroidManifest.xml` to include relevant flags:

- In the `<application>` or specifically for `CompetitiveModeActivity` entry, add `android:enableOnBackPressedCallback="true"` (ensures predictive back animations are not opted-out ²¹). This might be the default, but we include it for clarity.
- If using transitions across activities, set `<activity android:windowAnimationStyle="@style/YourTransitionStyle">` or simply rely on our code. Notably, if using Material transitions, we don't necessarily need manifest entries for them aside from the general `windowContentTransitions`.
- Ensure the activity has `android:exported="true"` if deep links etc., but that's unrelated to UI.

With these deliverables implemented, the Competitive Mode will transform into a **cutting-edge, elegant interface**. The chessboard remains the focal point, but now floats within a visually cohesive scene: frosted glass panels display player info and controls, seeming to hover above a rich blurred background. Android 15's features not only enhance aesthetics (via blur and Material3 theming) but also improve UX (with edge-to-edge content and smooth navigation gestures ¹⁰ ²²). This redesign balances **performance and beauty** – using GPU-accelerated effects and modern components so even complex visuals run efficiently on a flagship device. All the while, we respect the existing architecture (Views and XML layouts), upgrading it incrementally to deliver a polished competitive chess experience.

Sources: The implementation is informed by Android's official guidelines on blur effects, Material Design, and navigation for API 31+ and API 35 features ¹ ⁸ ¹⁰ ¹⁵. These ensure our design choices follow best practices and utilize the latest Android capabilities to the fullest.

1 2 Migrate from RenderScript | Views | Android Developers

<https://developer.android.com/guide/topics/renderscript/migrate>

3 7 Implementing Glassmorphism in Android App | by Anang Kurniawan | Staffinc Tech | Medium

<https://medium.com/sampingan-tech/implementing-glassmorphism-in-android-app-e73a2fd83b80>

4 5 6 8 Window blurs | Android Open Source Project

<https://source.android.com/docs/core/display/window-blurs>

9 Material Design Components for Android 1.10.0

<https://m3.material.io/blog/android-stable-release-1-10-0>

10 11 12 13 14 Display content edge-to-edge in views | Views | Android Developers

<https://developer.android.com/develop/ui/views/layout/edge-to-edge>

15 16 17 18 21 22 Add support for the predictive back gesture | App architecture | Android Developers

<https://developer.android.com/guide/navigation/custom-back/predictive-back-gesture>

19 20 MaterialContainerTransformSharedElementCallback | API reference | Android Developers

<https://developer.android.com/reference/com/google/android/material/transition/platform/MaterialContainerTransformSharedElementCallback>